Python Functions

Class Notes — Reusable, Modular Code

Instructor: Tanishq Tyagi | Python Programming

Part 1 — Why Functions Matter?

As programs grow, repeating the same logic in multiple places leads to messy, error-prone code. **Functions** solve this by letting you wrap a block of code into a named, reusable unit. Instead of writing the addition logic three times across 300 lines, you define it once and call it whenever you need it.

> **INFO** Think of a function like a recipe: write the steps once, then cook the dish as many times as you like — without rewriting anything.

The problem without functions:

```
# Line 10
num1 = 10; num2 = 20
print(f'{num1} + {num2} = {num1+num2}')

# Line 200 — same logic again
num1 = 4; num2 = 6
print(f'{num1} + {num2} = {num1+num2}')

# Line 300 — and again ...
num1 = 23; num2 = 45
print(f'{num1} + {num2} = {num1+num2}')
```

With a function — one definition, called anywhere:

```
def add(num1, num2):
total = num1 + num2
print(f'{num1} + {num2} = {total}')
return total

add(10, 20) # Line 10
add(4, 6) # Line 200
add(23, 45) # Line 300
```

Part 2 — Benefits of Functions

1. Code Reusability

Write the logic once; call it many times. If the logic changes, you update **one place** and every caller automatically gets the fix.

2. Modularity

Break a large program into small, focused functions. Each function does one thing well — this makes it easy to read, test, and debug individual pieces independently.

3. DRY Principle — Don't Repeat Yourself

DRY is a core software engineering principle: every piece of knowledge or logic should exist in exactly one place. Functions are the primary tool for achieving DRY in Python.

> **NOTE** Wet code (Write Everything Twice) is the enemy of maintainability. If you copy-paste code, ask yourself: 'Can this be a function?'

Part 3 — Defining and Calling Functions

Syntax:

```
def function_name(parameters):
# body — indented block
statement_1
statement_2
return value # optional
```

Keyword	Purpose
def	Declares a new function
( )	Holds zero or more parameters
:	Ends the function header; body must be indented
return	Sends a value back to the caller (optional)

Example from class:

```
def add(num1, num2):
total = num1 + num2
print(f'{num1} + {num2} = {total}')
return total

# Calling the function
a = 10
b = 20
result = add(a, b) # Output: 10 + 20 = 30
```

> **TIP** A function is DEFINED once (with def) but can be CALLED any number of times. Defining it does not execute any code — the body only runs when you call it.

Part 4 — Parameters and Arguments

Parameters are the variable names listed in the function definition. **Arguments** are the actual values passed when the function is called.

Term	Where	Example
Parameter	Function definition	def greet(name):
Argument	Function call	greet('Tanishq')

Positional arguments:

Arguments are matched to parameters in the order they are written.

```
def print_list_elements(list_data):
for item in list_data:
print(item)
print('Printed list elements!')

names = ['Shachi', 'Mourya', 'Vibhore']
print_list_elements(names) # positional — names goes into list_data
```

Default parameters:

A parameter can have a **default value**. If the caller doesn't pass that argument, the default is used.

```
def greet_user(name='User'):
print(f'Welcome aboard! {name}')

greet_user() # Output: Welcome aboard! User
greet_user('Tanishq') # Output: Welcome aboard! Tanishq
```

> **NOTE** Default parameters must come AFTER non-default parameters in the definition. def greet(name, greeting='Hello') is valid; def greet(greeting='Hello', name) is NOT.

Part 5 — Keyword Arguments

When calling a function you can specify arguments by their parameter name (keyword). This lets you pass them in any order.

```
def print_name_age(name, age):
print(f'Name: {name}, Age: {age}')

# Positional — order matters
print_name_age('Tanishq', 4)
```

```
# Keyword — order doesn't matter
print_name_age(age=4, name='Tanishq') # same result!
```

TIP Keyword arguments make function calls self-documenting. Seeing age=4, name='Tanishq' is instantly readable without looking up the definition.

Part 6 — Return Statements

Single return value:

Use **return** to send a computed value back to the caller. Code after return is never executed — return exits the function immediately.

```
def sum_of_list(nums):
total = 0
for n in nums:
total += n
return total # sends total back

result = sum_of_list([1, 2, 3, 4, 5])
print(result) # 15
```

Multiple return values:

Python allows returning more than one value — they are packed into a **tuple** automatically and can be unpacked on the left-hand side.

```
def calculate_sum_and_avg(nums):
total = sum(nums)
avg = total / len(nums)
return total, avg # returns a tuple

nums = [1, 2, 3, 4, 5, 6]
s, a = calculate_sum_and_avg(nums) # tuple unpacking
print(f'Sum={s}, Avg={a}') # Sum=21, Avg=3.5
```

Chaining functions using return values:

```
def sum_of_list_elements(nums):
total = 0
for n in nums: total += n
return total
```

```
def find_avg(total, n):
return total / n

def calculate_sum_and_avg(nums):
s = sum_of_list_elements(nums)
a = find_avg(s, len(nums))
return s, a

nums = [1, 2, 3, 4, 5, 6]
s, a = calculate_sum_and_avg(nums)
print(f'Sum={s}, Avg={a}') # Sum=21, Avg=3.5
```

INFO Without return, Python functions implicitly return None. Always return a value when the caller needs to use the result.

Part 7 — Variable Scope

The **scope** of a variable determines where in the code that variable can be accessed.

Global Variables:

Defined at the top level of the script (outside any function). Accessible from anywhere in the file.

Local Variables:

Defined inside a function. Only accessible within that function. They are created when the function is called and destroyed when it returns.

```
a = 10 # global variable

def print_a():
c = 10 # local variable – only exists inside print_a()
print(f'Value of c is: {c}')

def print_c():
# print(c) # ERROR! c is not visible here
print('c is not accessible here')

print_a() # fine
print_c() # fine
# print(c) # ERROR – c is local to print_a
```

TIP Rule of thumb: prefer passing values as arguments rather than relying on global variables. It makes functions easier to test and reuse.

Part 8 — Scope Resolution: LEGB Rule

When Python encounters a variable name it searches for its definition in this fixed order:

Letter	Scope	Description
L	Local	Inside the current function
E	Enclosing	Inside any enclosing (outer) function — for nested functions
G	Global	At the top level of the module / script
B	Built-in	Python's own built-ins: len, print, range, ...

Class example — LEGB in action:

```
name = 'Global Tanishq' # G — global

def outer_function():

message = 'Hello from enclosing scope' # E — enclosing

# name = 'Enclosing Tanishq' # (commented out)

def inner_function():

# name = 'Inner Tanishq' # L — local (commented out)

print(f'name: {name}, message: {message}')

# name -> not local, not enclosing -> found at GLOBAL

# message-> not local -> found at ENCLOSING

inner_function()

outer_function()

# Output: name: Global Tanishq, message: Hello from enclosing scope
```

Visualising LEGB search:

```
# Python checks in this order for every name:

#

# inner_function scope -> outer_function scope -> module scope -> builtins

# (L) (E) (G) (B)

#

# First match wins; if none found -> NameError
```

INFO Remember: L-E-G-B goes from innermost to outermost. Python stops at the first scope where the name is found. Uncommenting 'name = Inner Tanishq' inside inner_function would shadow the global name — LEGB would stop at L.

End of Class Notes — Python Functions | Tanishq Tyagi